

Final Project Report

담당 교수 :	박성용 교수님
학번 :	20221549
이름 :	김효림

0. specification

- CPU: Intel Core i7-1165G7 (Tiger Lake), base 2.80 GHz
- 물리 코어/논리 스레드: 4 코어 / 8 스레드(Hyper-Threading 2x)
- 캐시: L1d 48 KiB×4, L1i 32 KiB×4, L2 1.25 MiB×4, L3 12 MiB 공유
- RAM(WSL 가용): 7.8 GiB (Swap 2.0 GiB)
- OS / 커널: Ubuntu 22.04.5 LTS / 6.6.87.2-microsoft-standard-WSL2
- 컴파일러: g++ 11.4.0, 플래그 ``-std=c++17 -O2 -Wall -fopenmp``
- 미들웨어: gRPC 1.30.2, protobuf 3.12.4

1. Abstract & Problem Definition

(a) target application

LoL 랭크 매치 51,490 건 기반 챔피언 조합 분석 서비스로, 클라이언트가 조합을 질의하면 서버가 승률과 그에 대한 95% 부트스트랩 신뢰구간을 응답하는 시스템이다.

(b) computational bottlenecks

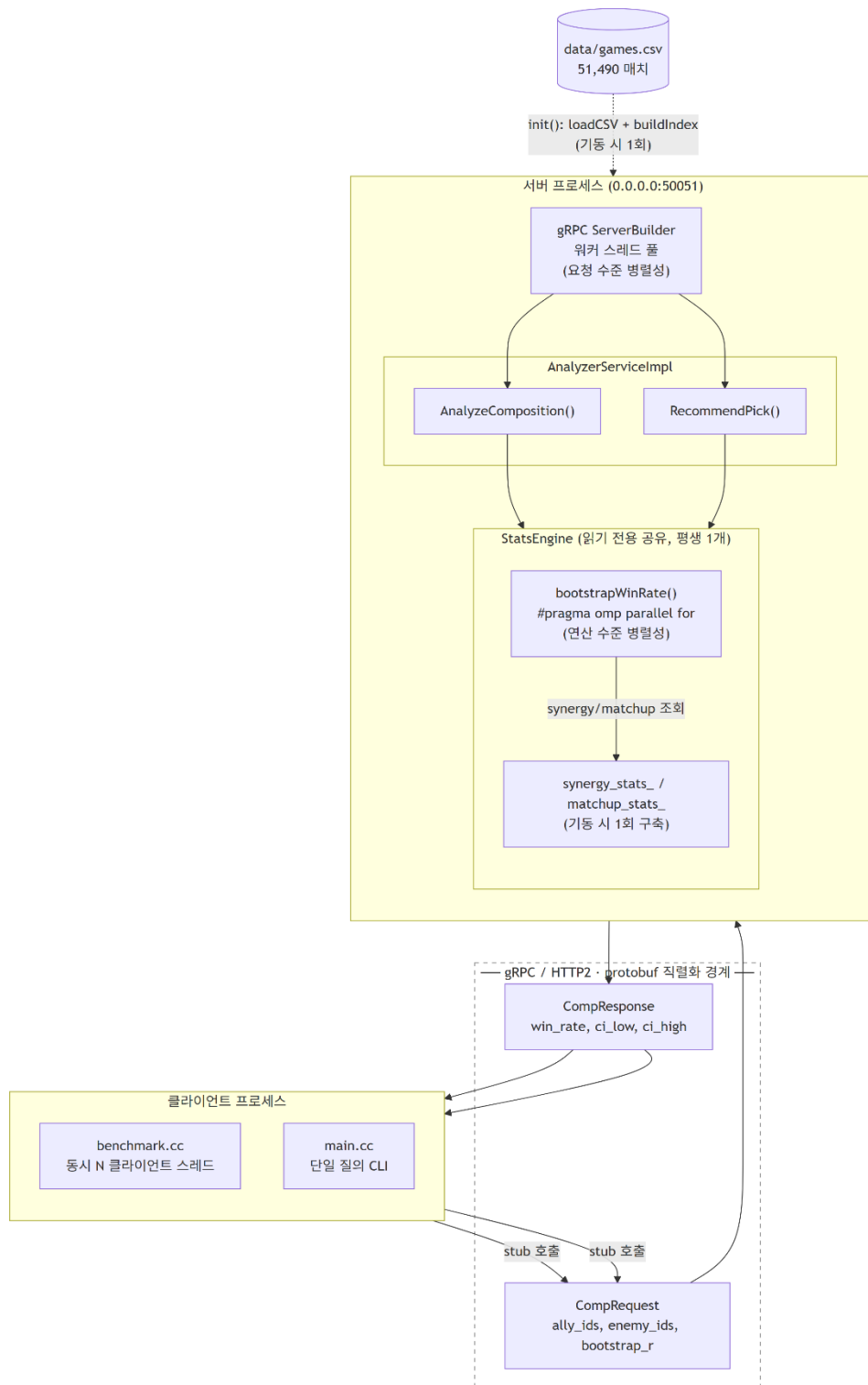
질의 1 건당 부트스트랩 R 회 리샘플 × 매치 51k 건 × 45 쌍 검사를 진행한다. R=1000 이면 약 23 억 회의 쌍 검사를 수행한다.

(c) Design rationale

- gRPC(미들웨어): 클라이언트가 조합을 질의하고 분석 결과를 돌려받는 요청-응답 워크로드이므로 동기 RPC 방식을 선택했다. 초기에 Kafka 도 고려했으나, 데이터가 기동 시 한 번 메모리에 적재된 뒤 새 이벤트의 연속 유입이 없어 스트림 처리 모델이 맞지 않으므로 gRPC 로 결정했다.
- OpenMP: 부트스트랩 리샘플은 작업량이 균일한 독립 반복이라 fork-join 데이터 병렬에 적합하다. 단일 노드에 데이터가 상주하므로, 프로세스 간 통신이나 데이터 분할이 필요한 MPI 는 오히려 오버헤드만 늘리므로 OpenMP 를 채택했다.
- 연산 강도 설계: 데이터가 작아 I/O 에서 병목이 발생하지 않으므로, 부트스트랩 R 로 CPU 연산량을 의도적으로 키워 가속 측정이 의미를 갖도록 했다.

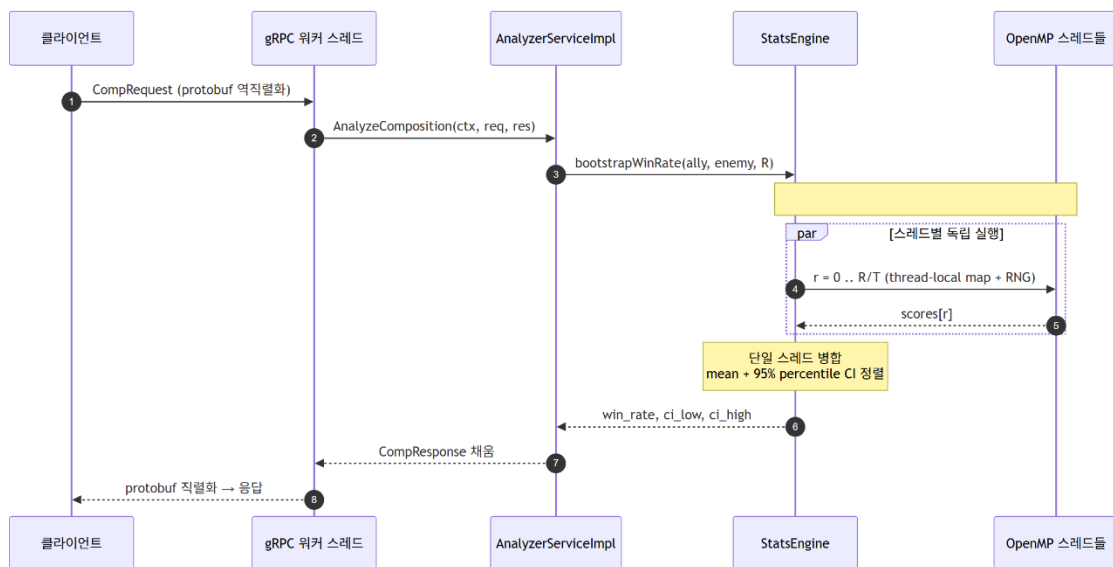
2. System Architecture Diagram

클라이언트 프로세스와 서버 프로세스가 gRPC 로 분리된 요청-응답 구조로, 서버 내부에서 요청 수준과 연산 수준의 두 병렬성이 겹친다. 전체 구조는 다음과 같다.



[그림 1] 전체 블록 다이어그램

- Data flow: 클라이언트가 챔피언 조합을 담은 **CompRequest** 를 stub 으로 전송하면, 서버의 gRPC 워커가 요청을 받아 **AnalyzerServiceImpl** 의 **AnalyzeComposition** 또는 **RecommendPick** 핸들러로 전달한다. 핸들러는 **StatsEngine** 을 호출해 부트스트랩 승률과 신뢰구간을 계산하고, 결과를 **CompResponse** 로 채워 클라이언트에 반환한다. 매치 데이터는 요청 경로가 아니라 기동 시 1 회 로드되어 인덱스로 상주한다.
- Network boundary: 클라이언트와 서버 사이의 경계는 gRPC 채널(HTTP/2, 기본 포트 50051)이며, 통신에 포함되는 모든 메시지는 protobuf 로 직렬화된다. 즉 [그림 1]의 점선 박스가 프로세스 간 통신과 직렬화 비용이 발생하는 지점이다.
- Middleware stage: gRPC 의 ServerBuilder 가 워커 스레드 풀을 구성해 들어오는 RPC 를 디스패치한다. stateless 서비스라 워커는 공유 인덱스를 읽기만 하므로 요청 간 lock 이 필요 없다.
- Thread/process organization: 서버는 단일 프로세스 안에서 두 층위로 스레드를 쓴다. gRPC 워커 풀이 동시 클라이언트 요청을 병렬 처리하고, 각 요청 처리 중 **bootstrapWinRate()**가 OpenMP 병렬 구간으로 진입해 부트스트랩 리샘플을 병렬 처리한다. **StatsEngine** 은 프로세스 수명 동안 1 개만 존재하며 읽기 전용으로 공유된다.
- Parallel acceleration stage: CPU-bound 연산은 **bootstrapWinRate()**의 리샘플 루프에 집중되며, 여기에 **#pragma omp** 로 병렬화가 적용된다. 기동 시 **buildIndex()**의 매치당 쌍 집계도 동일하게 OpenMP 로 병렬화된다.



[그림 2] 요청 처리 시퀀스

[그림 2]는 **AnalyzeComposition** 요청 1 건이 직렬화 → 워커 디스패치 → OpenMP 부트스트랩 → 병합·정렬 → 직렬화로 이어지는 흐름을 보여준다. 부트스트랩 단계에서 병목이 통신에서 CPU 로 이동하는 것을 확인할 수 있다.

[그림 1]과 [그림 2]에서 확인할 수 있듯이, 해당 프로젝트는 두 층위의 병렬성이 코어를 두고 경쟁한다. 따라서 워커 수와 OpenMP 스레드 수의 트레이드오프가 성능을 좌우한다.

	요청 수준 (gRPC 워커)	연산 수준 (OpenMP)
병렬 단위	클라이언트 요청	부트스트랩 리샘플 r
담당 주체	gRPC 워커 스레드 풀	OpenMP 스레드
공유 데이터	StatsEngine 인덱스 (읽기 전용)	games_ 리샘플 (읽기 전용)
동기화	없음 (무상태)	Thread-local map 병합 / per-thread RNG
측정 축	동시 클라이언트 수 → latency, throughput	OpenMP 스레드 수 → speedup

3. Implementation Details

(a) Middleware configuration (**server/main.cc**)

동기 gRPC 서버를 사용한다. **gRPC::ServerBuilder** 로 서비스를 등록하고 **BuildAndStart()** 후 **Wait()**로 대기한다. 동기 모델을 택했기 때문에 수신한 각 요청은 gRPC 런타임이 관리하는 워커 스레드 풀에 자동 디스패치되고, 각 핸들러는 응답을 채울 때까지 해당 워커를 점유한다. 별도 코드 없이 워커 풀이 동시 요청을 병렬 처리하므로 요청 수준 병렬성이 확보된다. 핸들러는 요청 데이터를 스레드 로컬 vector 로 복사해 처리하고, 공유 인덱스는 읽기만 하므로 워커 간 공유 가변 상태가 없어 안전하다.

이때 미들웨어 계층에서 요청 한 건의 연산량 상한을 지정해(`bootstrap_r = [0,2000]`) 과도한 R 이 워커를 장시간 점유하는 것을 방지한다.

(b) Serialization methods (**proto/analyzer.proto**)

Protobuf3(proto3 syntax)로 직렬화한다.

- **CompRequest**: repeated int32 ally_ids, enemy_ids, int32 bootstrap_r
- **CompResponse**: double win_rate / ci_low / ci_high

조합당 챔피언 id 5 개와 정수 1 개를 받으므로 요청 payload 는 수십 바이트 수준이고, 응답은 double 3 개로 구성된다.

(c) Batch sizes

OpenMP 기반 CPU 파이프라인으로 GPU 배치 개념은 없다. 대신 부트스트랩 리샘플 횟수 R 이 연산 배치 크기의 역할을 한다.

(d) Thread/Block structures

두 층위 스레드 구조를 갖는다.

- 요청 수준: gRPC 동기 서버가 런타임에서 관리하는 워커 스레드 풀이며, stateless 핸들러가 공유 StatsEngine 을 읽기만 한다.
- 연산 수준(stats_engine.cc): bootstrapWinRate()가 #pragma omp parallel 로 여러 스레드를 생성해 부트스트랩 리샘플을 나눠 처리한다. parallel 로 감싼 구문 내부에서 스레드마다 독립 자원을 준비한 후, #pragma omp for 로 R 개의 리샘플을 스레드들에 나눠 각자 scores[r]에 결과를 쓴다.

```
#pragma omp parallel
{
    uint32_t seed = base_seed ? *base_seed : random_device{}();
    mt19937 rng(seed ^ (uint32_t)(omp_get_thread_num() * 2654435761u));
    uniform_int_distribution<int> dist(0, n - 1);

    unordered_map<uint64_t, PairStat> syn_r, mat_r;
    syn_r.reserve(syn_keys.size() * 2);
    mat_r.reserve(mat_keys.size() * 2);

    #pragma omp for schedule(static)
    for (int r = 0; r < R; ++r) {
        syn_r.clear();
        mat_r.clear();
        for (uint64_t k : syn_keys) syn_r[k] = {};
        for (uint64_t k : mat_keys) mat_r[k] = {};
    }
}
```

i) per-thread RNG: 각 스레드가 $\text{seed} \wedge (\text{tid} \times 2654435761u)$ 로 고유 시드를 받는다. 스레드 간 난수열이 겹치지 않게 분리하면서, BOOTSTRAP_SEED 고정 시 (시드, 스레드 수)에 대해 결정적 재현이 가능하다.

ii) thread-local 집계 맵: 스레드별 syn_r, mat_r 을 잡고, 그 후 #pragma omp for schedule(static)로 R 개의 리샘플을 분배한다.

- 마지막으로 buildIndex()와 recommendPicks()는 반복이 단순 독립이기 때문에 #pragma omp parallel for schedule(static)만을 사용해 병렬화했다.

```

137 void StatsEngine::buildIndex() {
138     int n      = (int)games_.size();
139     int nthreads = omp_get_max_threads();
140
141     // Allocate one map per thread to avoid false-sharing / locking
142     vector<unordered_map<uint64_t, PairStat>> local_syn(nthreads);
143     vector<unordered_map<uint64_t, PairStat>> local_mat(nthreads);
144
145     #pragma omp parallel for schedule(static)
146     for (int i = 0; i < n; ++i) {
147         int tid = omp_get_thread_num();
148         const Game& g = games_[i];
149         bool t1_won = (g.winner == 1);
150
151
229     // scoreComposition only reads the (immutable) index, so this is race-free.
230     #pragma omp parallel for schedule(static)
231     for (int i = 0; i < m; ++i) {
232         vector<int> trial = ally_ids;
233         trial.push_back(candidates[i]);
234         scores[i] = scoreComposition(trial, enemy_ids);
235     }

```

(e) Synchronization methods

읽기는 공유, 쓰기는 thread-local 후 단일 스레드로 병합하는 방식을 채택하였다.

- buildIndex(stats_engine.cc:142-192) (인덱스 구축): 각 스레드가 자신의 맵 (local_syn[tid], local_mat[tid])에만 누적하고, 병렬 구간이 끝난 뒤 단일 스레드가 전역 맵으로 병합한다.
 - 대안 비교: 전역 맵 + mutex 는 매치당 45 회 x 51k = 수백만 번의 lock 획득으로 병렬의 의미가 사라진다.
- bootstrap (리샘플): 결과는 scores[r]에 작성하여 리샘플 r 마다 서로 다른 인덱스에 1 회만 쓰기를 진행한다. schedule(static)이 스레드 별로 연속 구간을 지정하므로, 두 스레드가 같은 캐시라인을 공유하는 건 구간 경계의 캐시라인 하나뿐이다. 게다가 각 scores[r]은 51k 매치를 도는 무거운 내부 루프 뒤 1 회만 기록되므로, 경계에서의 false sharing 빈도가 극히 낮아 사실상 무시 가능하다.
- Serial 구간: 리샘플 종료 후 평균 계산과 신뢰구간을 위한 sort 는 단일 스레드로 수행된다. 이 부분이 암달 한계의 원천이며, 4 절 thread scaling 에서 speedup 이 꺾이는 요인으로 분석한다.

(f) Parallelization strategies

- 부트스트랩은 리샘플(r) 단위로 병렬화한다. R 개의 리샘플은 독립적이고 작업량이 균일하다. 따라서 `schedule(static)`으로 분배하면 부하가 고르게 나뉘고 동적 스케줄링 오버헤드도 없다. 반대로 내부 51k 매치 루프나 쌍 단위로 쪼개면 리샘플마다 공유 맵으로의 reduction 이 필요해져 동기화 비용이 생긴다.
- 추천 질의는 후보 챔피언 단위로 병렬화한다. 각 후보의 `scoreComposition()`은 읽기 전용 인덱스만 참조하는 독립 계산이라 동기화가 필요 없다.
- `reserve()`로 rehash 를 방지한다. 리샘플 루프 진입 전, `syn_r` 과 `mat_r` 을 키 개수의 두 배로 예약하고, CSV 로드 시 `games_.reserve(55000)`로 재할당을 막는다. 핫루프 도중의 rehash 와 재할당 지연을 제거하기 위한 동작이다.

4. Performance Evaluation & Bottleneck Analysis

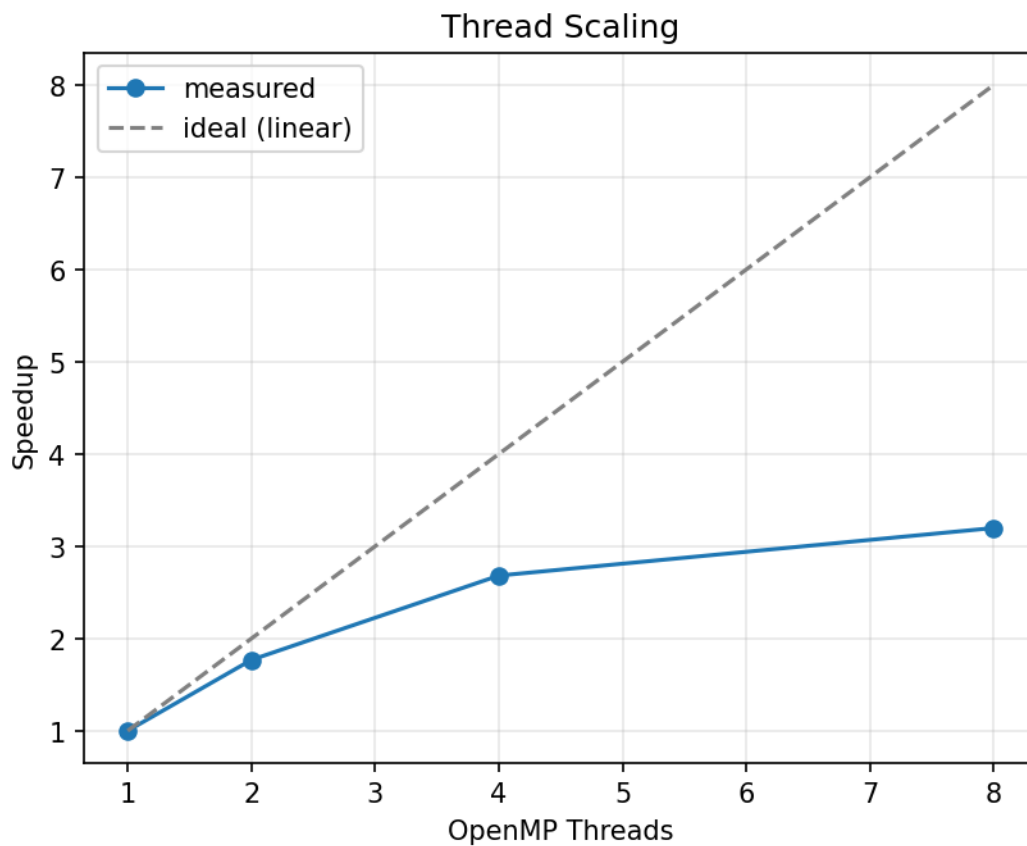
(a) evaluation

측정값을 오염시키는 요소를 하나씩 제거했다.

- 독립 연결(`benchmark.cc:62`): 동시 클라이언트 N 개는 각각 별도 스레드에서 독립 gRPC 채널과 stub 을 생성한다. 연결을 공유하지 않으므로 실제 동시 부하를 그대로 재현한다.
- 경합 없는 결과 수집: 각 스레드는 자신의 `results[i]` 칸에만 latency 를 기록하고, 측정이 끝난 뒤 단일 스레드가 병합한다.
- 동시 출발(`benchmark.cc:73`): 모든 스레드가 atomic 플래그에서 스핀 대기하다가, 준비가 끝나면 일제히 출발한다. 16 개 요청이 동시에 들이치는 순간을 정확히 측정하기 위해 관련 없는 overhead 를 없앴다.
- 워밍업 폐기: 스레드마다 처음 다섯 건은 버린다. 첫 요청에는 TCP 핸드셰이크와 HTTP/2 연결 수립 비용이 섞이므로, 정상 상태 latency 만 남긴다.
- 스레드 고정: 서버의 OpenMP 스레드를 물리코어에 할당한다. OS 스케줄러가 스레드를 코어 사이로 옮기며 생기는 캐시 오염과 측정 노이즈를 제거한다.
- WSL2 환경에서 측정하여 세션마다 상태가 달라 절대 latency 에 변동이 있다. 따라서 각 실험은 한 세션 안에서 변수 하나만 바꿔 측정하였으며 실험 간 절대값을 비교하지 않는다.

(b) Thread Scaling

클라이언트를 1 개로 고정하면 요청끼리 코어를 경쟁하지 않으므로, 변하는 것은 서버의 OpenMP 스레드 수 하나다. `OMP_NUM_THREADS` 는 서버 기동 시점에만 적용되어 스레드 값마다 서버를 재기동했고, 충분한 연산량 확보를 위해 `R=1000` 으로 측정했다.



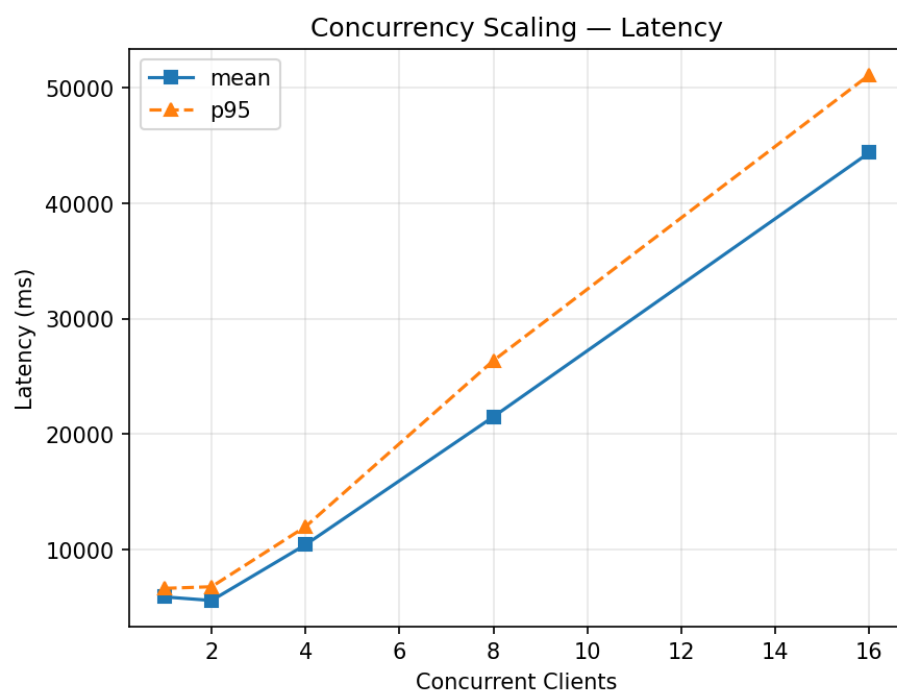
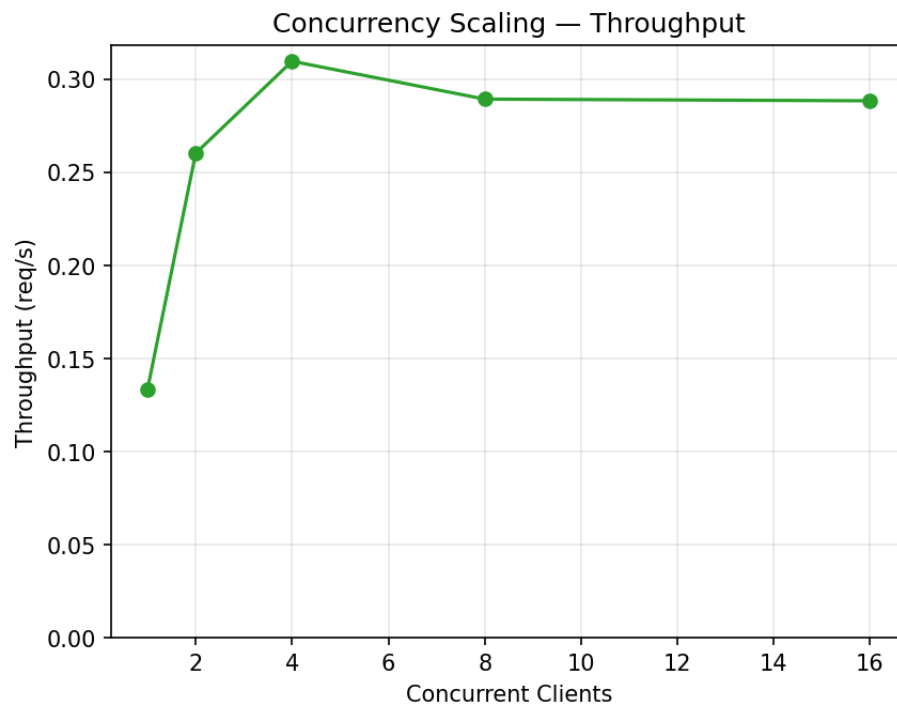
OMP	Mean (ms)	Speedup	효율
1	22,337	1.00x	100%
2	12,632	1.77x	88%
4	8,328	2.68x	67%
8 (SMT)	6,984	3.20x	40%

Speedup 이 선형에서 점점 벌어지며, 8 스레드에서 3.20x 에 그친다. 원인은 다음과 같다.

- Amdahl 한계: 부트스트랩 루프는 병렬화되지만, CSV 인덱싱과 결과 병합, 정렬, protobuf 직렬화 같은 Serial component 가 남는다. 이 직렬 비율이 speedup 의 상한을 결정한다.
- 물리코어 한계(SMT): 실험은 CPU 가 4 물리 코어 / 8 논리 스레드인 환경에서 진행됐다. 스레드의 수가 4 개에서 8 개로 넘어가는 구간에서 효율 가속이 완만해지는 것은, 5~8 스레드가 새 물리코어를 점유하는 것이 아니라 하이퍼스레딩으로 코어 내부 자원을 나눠 쓰기 때문이다.

(c) Concurrency Scaling

서버를 전체 코어(OMP=전체)로 한 번만 띄우고 OpenMP 설정을 고정한 채, 동시 클라이언트 수만 늘렸다.



동시 클라이언트	Throughput	평균 latency	P99	구간
1	0.133 /s	5.9 s	6.6 s	여유
2	0.260 /s	5.6 s	6.8 s	여유
4	0.309 /s	10.4 s	12.3 s	포화점
8	0.289 /s	21.5 s	27.1 s	큐잉
16	0.288 /s	44.4 s	54.4 s	큐잉

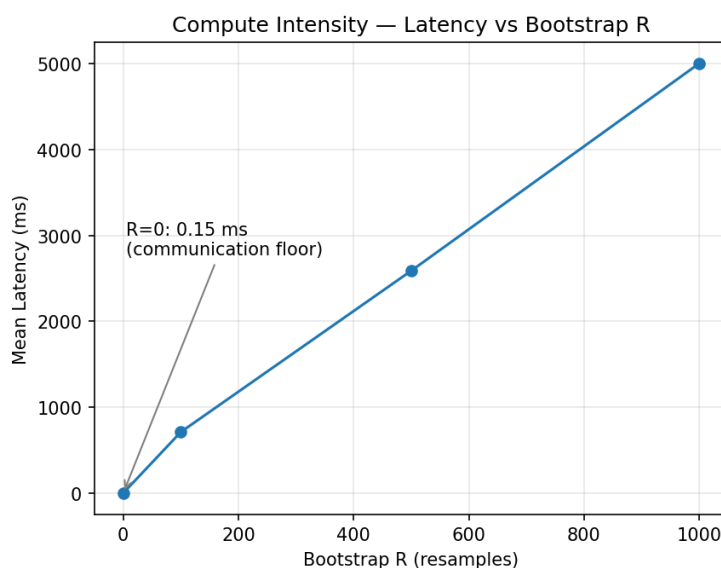
Throughput 은 4 클라이언트에서 0.309 /s 로 정점을 찍고 평평해지며, 그 이후 latency 는 거의 선형으로 증가한다.

- 1 → 2 클라이언트: throughput 이 거의 2 배인데 latency 는 그대로다. 단일 요청이 모든 코어를 다 채우지 못해 여유가 있었고, 두 번째 요청이 그 빈 용량을 채운 것이다.
- 4 클라이언트: 코어가 가득 차 포화 상태가 되었다.
- 8, 16 클라이언트: throughput 은 더 오르지 않고 latency 만 비례해 증가한다. 요청들이 바로 처리되지 않고 큐에 대기하게 된다.

서버는 요청 하나를 처리할 때 이미 OpenMP 로 전체 코어를 쓰러 한다. 따라서 동시 요청이 4 개를 넘으면, 각 요청이 모두 전체 코어를 원하면서 요청 수준 병렬성(gRPC 워커)과 연산 수준 병렬성(OpenMP)이 같은 물리코어를 두고 경쟁한다. 이것이 포화 이후 latency 가 선형으로 늘어나는 원인이다.

(d) Compute Intensity

클라이언트를 1 개로 고정하고 부트스트랩 R 만 0 에서 1000 으로 키웠다. R=0 은 통신 비용을 측정하기 위해 실험했다.



R	평균 latency	의미
0	0.149 ms	통신 비용 (직렬화 + 전송 + 역직렬화 + 디스패치)
100	712 ms	CPU 연산 시작
500	2,590 ms	통신 비중 0.006%
1000	5,004 ms	R=500 대비 x1.93

R=0 (0.149 ms)에서 R=100(712 ms)으로 가는 순간 latency 가 약 4,800 배 뛰고, 이후는 R 에 거의 완벽히 비례한다.

- R=0 의 0.149 ms 는 gRPC 왕복과 protobuf 직렬화의 오버헤드다. 메시지 marshaling 이 요청 0.05 μ s, 응답 0.07 μ s 수준임을 확인했으므로, 이 0.149 ms 중 직렬화 자체의 몫은 무시 가능하고 대부분이 RPC 프레임링과 loopback 전송에서의 코스트임을 알 수 있다.

- R 을 늘리면 latency 가 통신의 수천 배가 된다. 병목이 통신에서 CPU 로 이동하는 지점이며, 이것이 프로젝트에서 OpenMP 가속이 의미를 갖는 이유다.

(e) 병목 분석

세 실험이 가리키는 병목을 정리하면 다음과 같다.

병목	관련 실험	어떻게 나타나는가
Serialization / 네트워크	Intensity R = 0, 실험 4	0.149 ms (R=500 의 0.006%)
CPU 연산	Intensity R $\uparrow$, Thread scaling	Latency 가 R 에 선형, 병렬화로 가속되는 병목
Amdahl 직렬 구간	Thread scaling	병합, 정렬, 직렬화가 speedup 상한을 4x 아래로 제한
물리코어 한계(SMT)	Thread scaling	효율 67%에서 40%로 하락
두 층위 경쟁	Concurrency	포화 이후 latency 선형 증가

부트스트랩을 켜는 순간 시스템은 통신 바운드에서 CPU 바운드로 전환되며, 이 CPU 병목을 OpenMP 로 가속하되 그 이득은 Amdahl 직렬 구간과 물리코어 수에 의해 제한된다. 또한 요청 수준과 연산 수준의 병렬성이 같은 코어를 공유하므로, 최적 처리량은 둘의 곱이 물리코어 수를 넘지 않는 지점에서 형성된다.